12 TESTING E VALUTAZIONE DEI PROGRAMMI

OBIETTIVI

- Introdurre i diversi tipi di errori che possono verificarsi nella programmazione.
- Mostrare come usare le eccezioni per intercettare comportamenti difettosi.
- Applicare tecniche di programmazione difensiva.
- Spiegare come testare singole parti di una soluzione tramite il test unitario.
- Introdurre gli aspetti emergenti di una soluzione e come testarli.
- Testare l'intero programma tramite test di sistema e di accettazione.
- Mostrare metodi per individuare errori usando logging e debugger.

INTRODUZIONE AL TESTING E ALLA VALUTAZIONE DEI PROGRAMMI

Valutare un programma significa rispondere a due domande:

1. **Funziona senza errori?** → Questo viene verificato attraverso il testing.
2. **Funziona bene?** → Qui si valuta stabilità, prestazioni, sicurezza e altri aspetti.

Poiché lavoriamo con codice, **il testing può essere automatizzato**, scrivendo programmi che testano altri programmi. Questo capitolo mostrerà come fare test automatici in Python.

PREVEDERE I BUG

Molti bug si introducono già durante la **progettazione** di un programma, non solo durante la scrittura del codice.

💡 **Regola d'oro: tutti i programmi non banali contengono bug, anche quelli scritti da esperti.**

Errori sintattici vs. errori semantici

Tipo di errore	Descrizione	Esempio
Errore sintattico	Il codice non rispetta le regole del linguaggio.	class VendingMachine (manca :)
Errore semantico	Il codice è scritto correttamente, ma ha un significato sbagliato.	x = y / z (se z = 0, si ha un errore matematico)

📌 **Gli errori sintattici vengono rilevati prima dell'esecuzione, mentre gli errori semantici compaiono solo durante l'esecuzione.**

EVITARE I DIFETTI

Dal momento che alcuni errori emergono solo in fase di esecuzione, possiamo adottare strategie per gestirli.

Gestione delle eccezioni

Python usa il blocco try-except per intercettare gli errori.

python

CopiaModifica

```
try:
    x = 10 / 0  # Errore: divisione per zero
except ZeroDivisionError:
    print("Errore: divisione per zero.")
```

📌 **Se il codice nel blocco try genera un errore, il controllo passa al blocco except.**

Possiamo gestire **tipi diversi di eccezioni** in modo separato:

python

CopiaModifica

```
try:
    connection = CarSensor.create_connection()
    e = m / g  # Possibile divisione per zero
except ZeroDivisionError:
    print("Errore: consumo carburante segnalato come 0.")
except SensorUnreachableError:
    print("Errore: sensore auto non raggiungibile.")
```

Possiamo anche aggiungere un **blocco else** per eseguire codice solo se **non ci sono errori**:

python

CopiaModifica

```
try:
    e = m / g
except ZeroDivisionError:
```

```
    print("Errore.")
else:
    print(f"Efficienza: {e} mpg")
```

E un **blocco finally**, che viene sempre eseguito:

python

CopiaModifica

```
finally:
    connection.close()  # Chiude la connessione, anche in caso di errore
```

PROGRAMMAZIONE DIFENSIVA

La **programmazione difensiva** evita errori **prima che si verifichino**.

Esempio: una funzione che converte Celsius in Fahrenheit, ma verifica prima che il valore sia valido.

python

CopiaModifica

```
def celsius_to_fahrenheit(celsius):
    if celsius < -273:
        raise ValueError("Temperatura inferiore allo zero assoluto!")
    return celsius * 1.8 + 32
```

📌 **Se il valore è impossibile, la funzione interrompe immediatamente l'esecuzione.**

VERIFICA E VALIDAZIONE

✓ **Verifica** → Abbiamo costruito il prodotto nel modo giusto?
✓ **Validazione** → Abbiamo costruito il prodotto giusto?

💡 **Esempio**: Un ingegnere costruisce un ponte perfetto, ma se il cliente voleva un ponte ferroviario invece di uno pedonale, il progetto fallisce la validazione!

📌 **Il testing verifica aspetti tecnici. La validazione coinvolge l'utente finale.**

TESTARE LE PARTI DEL PROGRAMMA

Python ha un modulo integrato per i **test unitari**, chiamato unittest.

Esempio di test:

python

CopiaModifica

```
import unittest


class TestMathOperations(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(2 + 2, 4)


if __name__ == "__main__":
    unittest.main()
```

📌 **Se tutti i test passano, l'output sarà "OK". Se un test fallisce, Python mostrerà un messaggio d'errore.**

TESTARE L'INTERO SISTEMA

Testing del sistema

Testiamo il programma completo, verificando che tutte le parti funzionino insieme.

Esempio: test di login utente.

python

CopiaModifica

```
import unittest


class TestLogin(unittest.TestCase):
    def test_login(self):
        ui = UserInterface()
        ui.do_login(username="kirk", password="enterprise")
        self.assertEqual(ui.get_displayed_message(), "Login accettato.")
```

```
if __name__ == "__main__":
    unittest.main()
```

Testing di accettazione

Il cliente testa il programma per vedere se soddisfa le sue esigenze.

📌 **Esempio di strumento per i test di accettazione:**
Behave → Permette di scrivere test in linguaggio naturale.

python

CopiaModifica

```
from behave import *

@given("una macchina è pronta")
def step_impl(context):
    context.machine = VendingMachine()

@when("inserisco una moneta")
def step_impl(context):
    context.machine.insert_coin(0.50)

@then("il display mostra il valore della moneta")
def step_impl(context):
    assert context.machine.display == "0.50"
```

DEBUGGING

📌 **Strumenti di debugging**

1. **Logging** → Registra messaggi per capire cosa è successo prima di un crash.
2. **Debugger (pdb)** → Permette di eseguire il codice passo dopo passo.

Esempio di **logging**:

python

CopiaModifica

```
import logging

logging.basicConfig(filename="log.txt", level=logging.DEBUG)

logging.debug("Questo è un messaggio di debug")
```

Esempio di **uso del debugger**:

python

CopiaModifica

```
import pdb


x = 1

pdb.set_trace()  # Pausa l'esecuzione

y = 2

z = x + y

print(z)
```

CONCLUSIONE

- **Il testing aiuta a scoprire bug e migliorare la qualità del codice.**
- **Un buon programmatore anticipa i problemi e scrive codice difensivo.**
- **Strumenti come unittest, logging e pdb aiutano a individuare gli errori.**